

D2: Inductive Types and Classes

Constructors, recursion, induction, structures, and type classes

Summer School on Lean

July 14, Tuesday

Learning Goals

- read simple inductive, structure, and class declarations;
- predict the cases of a pattern match from the constructors of a type;
- recognize the shape of recursive definitions and induction proofs;
- use basic examples of `Option`, `List`, `Fin`, and `Multiset`;
- understand what instance search and deriving are for.

Four-Part Outline

Part I	Inductive Motivation and basic inductive declarations
Part II	Recursion and Induction Pattern matching, recursion, cases, and induction
Part III	Structures and APIs structure, Option, List, Fin, Multiset
Part IV	Type Classes class, instance, deriving, and an integrated expression example

Timing

Time	Topic
0–35 min	Part I: Inductive
35–80 min	Part II: Recursion and Induction
80–100 min	Break
100–145 min	Part III: Structures and APIs
145–180 min	Part IV: Type Classes

Data Built from Constructors

Many Lean types are described by listing their constructors:

- `Bool`: `true`, `false`
- `Nat`: `zero`, `succ`
- `Option alpha`: `none`, `some a`
- `List alpha`: `[]`, `x :: xs`

Knowing how values are built tells us how to compute with them and how to prove statements about them.

```
inductive Weekday where
  | mon | tue | wed | thu | fri | sat | sun
  deriving Repr, BEq, DecidableEq
```

- Weekday is a new type with seven constructors.
- The constructors are the only ways to build a Weekday.
- deriving generates printing and equality automatically.

Reading an Inductive Declaration

When you encounter an `inductive` declaration, ask:

- ① What is the new type?
- ② What are the constructors?
- ③ Which constructors carry data?
- ④ Which constructors are recursive?

These four answers predict the shape of pattern matches, recursive definitions, and induction proofs over the type.

Live Lean: `code/D2_Inductive.lean`

Constructors Are Functions

```
inductive MyNat where  
  | zero : MyNat  
  | succ : MyNat -> MyNat
```

- zero is already a natural number.
- succ needs a smaller natural number before it produces a new one.
- Use #check to inspect constructor types.

Parameters vs. Constructor Arguments

```
inductive MyOption (alpha : Type) where  
  | none : MyOption alpha  
  | some : alpha -> MyOption alpha
```

- alpha is a parameter of the whole family of types.
- some stores one particular value of type alpha.
- Examples: MyOption Nat, MyOption String.

Recursive Constructors

```
inductive MyNat where
  | zero : MyNat
  | succ : MyNat -> MyNat

inductive MyList (alpha : Type) where
  | nil : MyList alpha
  | cons : alpha -> MyList alpha -> MyList alpha
```

- Recursive constructors mention the type being defined.
- The recursive argument is the smaller object.
- This is why structural recursion and induction follow the same shape.

Addition on MyNat

```
def MyNat.add : MyNat -> MyNat -> MyNat
| m, zero => m
| m, succ n => succ (MyNat.add m n)
```

- The two equations follow the two constructors of MyNat.
- Recursion is on the second argument: `succ n` reduces to `n`.
- These are the familiar recursive equations $m + 0 = m$ and $m + (n + 1) = (m + n) + 1$.

```
inductive Tree (alpha : Type) where
  | leaf : alpha -> Tree alpha
  | node : Tree alpha -> Tree alpha -> Tree alpha
```

- A leaf stores one value.
- A node stores two subtrees of the same type.
- In the node case of an induction proof there are two induction hypotheses, one per subtree.

Part II: Recursion and Induction

Constructors are used in two more ways:

- pattern matching to inspect a value of an inductive type;
- structural recursion to define functions over recursive data.

Induction proofs then follow the same case structure as these definitions.

Pattern Matching

```
def isWeekend : Weekday -> Bool
| Weekday.sat => true
| Weekday.sun => true
| _ => false
```

- Pattern matching eliminates an inductive value.
- Each branch handles one possible constructor shape.
- The wildcard pattern `_` handles remaining cases.

Live Lean: `code/D2_PatternMatching.lean`

Matching on Option

```
def getWithDefault {alpha : Type}  
  (default : alpha) : Option alpha -> alpha  
  | none => default  
  | some a => a
```

- The none branch has no stored value.
- The some branch exposes the stored value.
- The return type is the same in both branches.

Recursive Definitions on Natural Numbers

```
def add (m : Nat) : Nat -> Nat
  | 0 => m
  | n + 1 => add m n + 1

def mul (m : Nat) : Nat -> Nat
  | 0 => 0
  | n + 1 => mul m n + m
```

- The recursion is on the second argument.
- The equations are ordinary recursive mathematical definitions.
- Powers can be defined in the same style.

Recursive Definitions on Lists

```
def myLength {alpha : Type} : List alpha -> Nat
| [] => 0
| _ :: xs => myLength xs + 1
```

- The recursive call is made on `xs`, a smaller list.
- Lean accepts this as structural recursion.
- The two branches match the two list constructors.

Map on Lists

```
def myMap {alpha beta : Type} (f : alpha -> beta) :  
  List alpha -> List beta  
| [] => []  
| x :: xs => f x :: myMap f xs
```

- The empty list maps to the empty list.
- The cons branch applies f to the head and recurses on the tail.
- The output list has the same length as the input list.

From Recursion to Induction

Data	Program	Proof
base constructor	base branch	base case
recursive constructor	recursive call	induction hypothesis

A function definition on an inductive type and an induction proof over that type share the same case structure.

Proof by Cases

```
example (b : Bool) : b = true \ / b = false := by
  cases b <;> simp
```

- cases splits a value according to its constructors.
- It is best for small or non-recursive inductive types.

Live Lean: `code/D2_InductionProofs.lean`

Induction on Lists

```
theorem append_nil_right {alpha : Type} (xs : List alpha) :  
  xs ++ [] = xs := by  
  induction xs with  
  | nil =>  
    rfl  
  | cons x xs ih =>  
    rw [ih]
```

- The base case is the empty list.
- The step case is $x :: xs$.
- The induction hypothesis is the theorem already known for xs .

Induction on Addition

```
theorem zero_add (n : Nat) : add 0 n = n := by
  induction n with
  | zero =>
    rfl
  | succ n ih =>
    calc
      add 0 (n + 1) = add 0 n + 1 := rfl
      _ = n + 1 := by rw [ih]
```

- The base case proves the statement for 0.
- The step case uses the induction hypothesis for n .
- This is the familiar proof of a recursive equation by induction.

20-minute break

Three topics:

- `structure`: an inductive type with one constructor and named fields;
- the standard inductive types `Option`, `List`, `Fin`, `Multiset`;
- how each one supports a different modeling choice.

Structures as Records

```
structure Vector2 where  
  x : Int  
  y : Int
```

- A structure bundles named fields.
- Values can be constructed with record syntax.
- Fields can be accessed with dot notation.

Live Lean: `code/D2_Structure.lean`

Operations on a Structure

```
def Vector2.add (u v : Vector2) : Vector2 :=  
  { x := u.x + v.x, y := u.y + v.y }  
  
def Vector2.dot (u v : Vector2) : Int :=  
  u.x * v.x + u.y * v.y
```

- `u.x` and `u.y` access coordinates.
- Record syntax constructs new vectors from field values.
- This mirrors ordinary coordinate definitions in linear algebra.

Bundling Data with a Property

```
structure PointOnDiagonal where
  x : Int
  y : Int
  onDiagonal : x = y
```

- Structures can contain data fields and proof fields.
- The field `onDiagonal` records a mathematical property.
- This pattern is common in formalized mathematics.

Inductive vs. Structure

Tool	Typical use	Shape
inductive	alternatives, recursive data	one or more constructors
structure	bundled fields	one constructor with named fields

Many mathematical objects in Lean are represented as structures.

Common Inductive Types

Type	Meaning	Typical use
<code>Option alpha</code>	a value or nothing	partial lookup
<code>List alpha</code>	finite ordered data	recursion and iteration
<code>Fin n</code>	a natural number below n	safe indexing
<code>Multiset alpha</code>	unordered data with multiplicity	finite combinatorics

Live Lean: `code/D2_CommonTypes.lean`

Option

```
def safeHead {alpha : Type} : List alpha -> Option alpha
| [] => none
| x :: _ => some x
```

- The return type signals that the lookup may have no result.
- Any caller has to handle both `none` and `some`.
- Partiality is recorded in the type, not in a side channel.

Common library operations on `List` `alpha`:

- `map`: transform each element;
- `filter`: keep elements satisfying a predicate;
- `foldl`, `foldr`: accumulate values from left or right;
- `length`, `++` (append), `mem`.

Each operation packages a standard recursive pattern over `nil` and `cons`.

```
#check Fin 3  
#check (0 : Fin 3)  
#check (2 : Fin 3)
```

- An element of `Fin n` is a natural number together with a proof that it is less than n .
- Writing `(3 : Fin 3)` is a type error.
- `Fin n` is the standard index type for vectors, matrices, and finite choices.

- A `List` `alpha` remembers order; `[1, 2, 1]` and `[1, 1, 2]` are different lists.
- A `Multiset` `alpha` forgets order but remembers multiplicity; the same two examples are equal as multisets.
- `Multiset` is defined in `mathlib` using `List` quotiented by permutation.

From Structures to Classes

- A `structure` bundles data and operations into one type.
- A `class` is a structure whose instances Lean searches for automatically.
- A parameter written as `[C alpha]` asks Lean to supply a `C alpha` instance at every call site.
- This is how `+`, `*`, and algebraic structure work uniformly across many types.

A Small Type Class

```
class HasNorm (alpha : Type) where
  norm : alpha -> Nat

def doubleNorm [HasNorm alpha] (x : alpha) : Nat :=
  2 * HasNorm.norm x
```

- `[HasNorm alpha]` is a type class parameter.
- Lean tries to synthesize the instance automatically.

Live Lean: `code/D2_Typeclass.lean`

```
structure Vector2 where
  x : Int
  y : Int

instance : HasNorm Vector2 where
  norm v := v.x.natAbs + v.y.natAbs
```

- An instance provides the data a class requires for a particular type.
- Once registered, the instance is found automatically wherever `[HasNorm Vector2]` is required.
- `Int.natAbs` sends an integer to its absolute value as a `Nat`.
- Use `#synth HasNorm Vector2` to ask Lean what it found.

Add and Mul as Classes

```
#synth Add Nat
#synth Add Int
#synth Mul Nat
#synth Mul Int

def addThree [Add alpha] (a b c : alpha) : alpha :=
  a + b + c
```

- + is the method of the class Add; * is the method of Mul.
- addThree works for any alpha with an Add instance.
- Built-in instances exist for Nat and Int; mathlib adds many more.

Mathlib: Algebraic Instances

```
import Mathlib

#synth Add Nat
#synth Add Int
#synth Add Rat
#synth Add Complex

#synth Ring Int
#synth Field Rat
#synth Field Complex
```

- Mathlib provides a rich algebraic hierarchy.
- Lean can find instances for familiar number systems.
- This is why generic algebraic theorems can apply broadly.

Live Lean: `code/D2_Typeclass_Mathlib.lean`

Classes for Mathematical Structure

Class	Purpose
Add, Mul	notation for operations
Zero, One	distinguished elements
Semigroup, Monoid	associative operations, identity
Group	inverse operation
Ring, CommRing	ring structure
Field	field structure

Operations vs. Laws

- `[Add alpha]` supplies only the operation `+` : `alpha -> alpha -> alpha`.
- It does not by itself give associativity, commutativity, or an identity.
- For those laws, use stronger classes: `AddCommMonoid alpha`, `Ring alpha`, `Field alpha`.
- A theorem stated with `[Ring R]` applies to every type for which Lean can find a `Ring` instance.


```
inductive Color where  
  | red | green | blue  
deriving Repr, BEq, DecidableEq
```

- deriving generates standard instances.
- It makes custom types usable in small examples immediately.

Expression Trees

```
inductive Expr where
  | const : Nat -> Expr
  | add : Expr -> Expr -> Expr
  | var : String -> Expr
  deriving Repr, BEq, DecidableEq
```

Syntax trees are naturally inductive.

Live Lean: `code/D2_IntegratedExample.lean`

Exercises

- Complete the guided holes in `D2_Exercises.lean`.
- Add a `mul : Expr -> Expr -> Expr` constructor to `Expr`.
- Extend `eval` and `vars` to the new constructor.
- Prove `nodeCount (mirror e) = nodeCount e` by induction on `e`.

Live Lean: `code/D2_Exercises.lean`